

WEEK-4

Task 1:

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <ctype.h>
#define MAX_INPUT 1024
#define MAX_ARGS 100
int check_quotes(const char *input)
{
    int single = 0;
    int doubleq = 0;
    int escape = 0;
    for (int i = 0; input[i] != '\0'; i++)
    {
        char c = input[i];
        if (escape)
        {
            escape = 0;
            continue;
        }
        if (c == '\\')
        {
            escape = 1;
        }
        else if (c == '"' && !doubleq)
        {
            single = !single;
        }
        else if (c == "'" && !single)
        {
            doubleq = !doubleq;
        }
    }
    if (single)
    {
        printf("Syntax Error: Unmatched single quote.\n");
        return 0;
    }
    if (doubleq)
```

```

    {
        printf("Syntax Error: Unmatched double quote.\n");
        return 0;
    }
    return 1;
}

void add_char(char *token, int *pos, char c)
{
    if (*pos < MAX_INPUT - 1)
    {
        token[*pos] = c;
        (*pos)++;
    }
}

int tokenize(char *input, char *args[])
{
    static char tokens[MAX_ARGS][MAX_INPUT];
    int argc = 0;
    int pos = 0;

    int single = 0;
    int doubleq = 0;
    int escape = 0;
    for (int i = 0; input[i] != '\0'; i++)
    {
        char c = input[i];
        if (escape)
        {
            add_char(tokens[argc], &pos, c);
            escape = 0;
            continue;
        }
        if (c == "\\")
        {
            escape = 1;
            continue;
        }
        if (c == "\"" && !doubleq)
        {
            single = !single;
            continue;
        }
        if (c == "'" && !single)
        {
            doubleq = !doubleq;
            continue;
        }
        if (isspace((unsigned char)c) && !single && !doubleq)

```

```

    {
        if (pos > 0)
        {
            tokens[argc][pos] = '\0';
            args[argc] = tokens[argc];
            argc++;
            if (argc >= MAX_ARGS - 1)
                break;
            pos = 0;
        }
    }
    else
    {
        add_char(tokens[argc], &pos, c);
    }
}

if (pos > 0)
{
    tokens[argc][pos] = '\0';
    args[argc] = tokens[argc];
    argc++;
}
args[argc] = NULL;
return argc;
}

void display_arguments(char *args[], int argc)
{
    printf("\n--- Parsed Arguments ---\n");
    for (int i = 0; i < argc; i++)
    {
        printf("Argument %d: %s\n", i + 1, args[i]);
    }
    printf("-----\n");
}

void execute_command(char *args[], int argc)
{
    pid_t pid;
    int status;
    pid = fork();
    if (pid < 0)
    {
        perror("fork");
        return;
    }
    if (pid == 0)
    {
        printf("Child Process PID: %d\n", getpid());
    }
}

```

```

        execvp(args[0], args);
        perror(args[0]);
        exit(127);
    }
    else
    {
        printf("Parent Process PID: %d\n", getpid());
        printf("Created Child PID: %d\n", pid);

        if (waitpid(pid, &status, 0) == -1)
        {
            perror("waitpid");
            return;
        }

        if (WIFEXITED(status))
        {
            printf("Child exited with status: %d\n",
                WEXITSTATUS(status));
        }
        else if (WIFSIGNALED(status))
        {
            printf("Child terminated by signal: %d\n",
                WTERMSIG(status));
        }
    }
}

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];
    printf("=====\n");
    printf("    MY CUSTOM SHELL\n");
    printf("=====\n");
    printf("Type 'exit' to quit.\n\n");
    while (1)
    {
        printf("myshell$ ");
        fflush(stdout);
        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            printf("\n");
            break;
        }
        input[strcspn(input, "\n")] = '\0';
        if (strlen(input) == 0)
        {
            printf("Empty command. Please enter a command.\n");

```

```

        continue;
    }
    if (strcmp(input, "exit") == 0)
    {
        printf("Shell terminated.\n");
        break;
    }
    if (!check_quotes(input))
    {
        continue;
    }
    int argc = tokenize(input, args);
    if (argc == 0)
    {
        printf("Empty command. Please enter a command.\n");
        continue;
    }
    display_arguments(args, argc);
    execute_command(args, argc);
    printf("\n");
}
return 0;
}

```

```

root@LAPTOP-RJFD8EBT:~# process_status_save task.c
root@LAPTOP-RJFD8EBT:~# mkdir -p ~/SKILL_Week_4
root@LAPTOP-RJFD8EBT:~# cd ~/SKILL_Week_4
root@LAPTOP-RJFD8EBT:~/SKILL_Week_4# pwd
/root/SKILL_Week_4
root@LAPTOP-RJFD8EBT:~/SKILL_Week_4# nano myshell.c
root@LAPTOP-RJFD8EBT:~/SKILL_Week_4# ls
myshell.c
root@LAPTOP-RJFD8EBT:~/SKILL_Week_4# gcc myshell.c -o myshell
root@LAPTOP-RJFD8EBT:~/SKILL_Week_4# ls -l
total 28
-rwxr-xr-x 1 root root 16880 Aug 20 05:42 myshell
-rw-r--r-- 1 root root 5285 Aug 20 05:41 myshell.c
root@LAPTOP-RJFD8EBT:~/SKILL_Week_4# ./myshell
=====
MY CUSTOM SHELL
=====
Type 'exit' to quit.

myshell$ echo Hello

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello
-----
Parent Process PID: 520
Child Process PID: 523
Created Child PID: 523
Hello
Child exited with status: 0

myshell$ ls

--- Parsed Arguments ---
Argument 1: ls
-----
Parent Process PID: 520
Created Child PID: 524
Child Process PID: 524
myshell myshell.c
Child exited with status: 0

```

```

Child exited with status: 0

myshell$ pwd

--- Parsed Arguments ---
Argument 1: pwd
-----
Parent Process PID: 520
Created Child PID: 525
Child Process PID: 525
/root/SKILL_Week_4
Child exited with status: 0

myshell$ date

--- Parsed Arguments ---
Argument 1: date
-----
Parent Process PID: 520
Created Child PID: 528
Child Process PID: 528
Thu Aug 20 05:44:28 UTC 2026
Child exited with status: 0

myshell$ Enter

--- Parsed Arguments ---
Argument 1: Enter
-----
Parent Process PID: 520
Created Child PID: 529
Child Process PID: 529
Enter: No such file or directory
Child exited with status: 127

```

Task 2:

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

```
myshell$ echo 'Hello World'

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World
-----
Parent Process PID: 520
Created Child PID: 625
Child Process PID: 625
Hello World
Child exited with status: 0

myshell$ echo 'Hello Linux World'

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello Linux World
-----
Parent Process PID: 520
Created Child PID: 626
Child Process PID: 626
Hello Linux World
Child exited with status: 0

myshell$ echo 'Hello $USER'

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello $USER
-----
Child Process PID: 627
Parent Process PID: 520
Created Child PID: 627
Hello $USER
Child exited with status: 0

myshell$ echo 'Hello $USER'

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello $USER
-----
Parent Process PID: 520
Created Child PID: 630
Child Process PID: 630
Hello $USER
Child exited with status: 0

myshell$
```

Example Output:

Case1:

myShell> echo Hello

Parsing Result:

Argument 1: echo

Argument 2: Hello

Child PID: 3210

Hello

Command execution completed.

Case2:

myShell> echo 'Hello World

Syntax Error: Unmatched single quote.

Task 3:

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

<pre>myshell\$ echo 'Hello \$USER' --- Parsed Arguments --- Argument 1: echo Argument 2: Hello \$USER ----- Parent Process PID: 520 Created Child PID: 630 Child Process PID: 630 Hello \$USER Child exited with status: 0 myshell\$ echo "Hello World" --- Parsed Arguments --- Argument 1: echo Argument 2: Hello World ----- Parent Process PID: 520 Created Child PID: 646 Child Process PID: 646 Hello World Child exited with status: 0 myshell\$ echo "Hello \$USER" --- Parsed Arguments --- Argument 1: echo Argument 2: Hello \$USER ----- Parent Process PID: 520 Created Child PID: 647 Child Process PID: 647 Hello \$USER Child exited with status: 0 myshell\$ echo "My home is \$HOME"</pre>	<pre>myshell\$ echo "Hello \$USER" --- Parsed Arguments --- Argument 1: echo Argument 2: Hello \$USER ----- Parent Process PID: 520 Created Child PID: 647 Child Process PID: 647 Hello \$USER Child exited with status: 0 myshell\$ echo "My home is \$HOME" --- Parsed Arguments --- Argument 1: echo Argument 2: My home is \$HOME ----- Parent Process PID: 520 Created Child PID: 654 Child Process PID: 654 My home is \$HOME Child exited with status: 0 myshell\$ echo "My shell is \$SHELL" --- Parsed Arguments --- Argument 1: echo Argument 2: My shell is \$SHELL ----- Parent Process PID: 520 Created Child PID: 657 Child Process PID: 657 My shell is \$SHELL Child exited with status: 0 myshell\$ echo "Hello World Syntax Error: Unmatched double quote. myshell\$ </pre>
--	--

Task 4:

To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

```

myshell$ echo Hello\ World
--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World
-----
Parent Process PID: 520
Created Child PID: 732
Child Process PID: 732
Hello World
Child exited with status: 0

myshell$ echo Hello\ Linux
--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello Linux
-----
Parent Process PID: 520
Created Child PID: 733
Child Process PID: 733
Hello Linux
Child exited with status: 0

myshell$ echo \$USER
--- Parsed Arguments ---
Argument 1: echo
Argument 2: $USER
-----
Parent Process PID: 520
Created Child PID: 736
Child Process PID: 736
$USER
Child exited with status: 0

myshell$ echo This\ is\ Linux
--- Parsed Arguments ---
Argument 1: echo
Argument 2: This is Linux
-----
Parent Process PID: 520
Created Child PID: 737
Child Process PID: 737
This is Linux
Child exited with status: 0

myshell$ echo Hello\!
--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello!
-----
Parent Process PID: 520
Created Child PID: 738
Child Process PID: 738
Hello!
Child exited with status: 0

myshell$ |

```

Task 5:

To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

```

mysHELL$ ls
--- Parsed Arguments ---
Argument 1: ls
-----
Parent Process PID: 520
Created Child PID: 746
Child Process PID: 746
mysHELL mysHELL.c
Child exited with status: 0

mysHELL$ ls -l
--- Parsed Arguments ---
Argument 1: ls
Argument 2: -l
-----
Parent Process PID: 520
Created Child PID: 747
Child Process PID: 747
total 28
-rwxr-xr-x 1 root root 16880 Aug 20 05:42 mysHELL
-rw-r--r-- 1 root root 5285 Aug 20 05:41 mysHELL.c
Child exited with status: 0

mysHELL$ ls -a
--- Parsed Arguments ---
Argument 1: ls
Argument 2: -a
-----
Parent Process PID: 520
Created Child PID: 748
Child Process PID: 748
. .. mysHELL mysHELL.c
Child exited with status: 0

mysHELL$ pwd
--- Parsed Arguments ---
Argument 1: pwd
-----
Parent Process PID: 520
Created Child PID: 749
Child Process PID: 749
/root/SKILL_Week_4
Child exited with status: 0

mysHELL$ ls @l
--- Parsed Arguments ---
Argument 1: ls
Argument 2: @l
-----
Parent Process PID: 520
Child Process PID: 761
Created Child PID: 761
ls: cannot access '@l': No such file or directory
Child exited with status: 2

```

Refer the following link to understand the Parsing and Lexical analysis
<https://www.geeksforgeeks.org/compiler-design/introduction-of-lexical-analysis/>